

OpenFlex 开源轮臂人形机器人使用说明书（基础篇）

[English](#) | [中文](#)

本文档详细描述 OpenFlex 整机机器人系统的硬件控制、VR 遥操作和各子系统使用方法。整机由 **四轮独立转向全向底盘 + 升降台 + 双 7 自由度机械臂 + 2 自由度头部** 组成，通过统一的 `ros2_control` 架构管理。

传感器、建图、定位、导航、VLA 数据采集/训练/推理及算法原理详见高级篇。

目录

- [第一章 构建工作空间](#)
 - [1.1 安装与构建](#)
- [第二章 快速开始](#)
 - [2.1 通过桌面 GUI 启动 OpenFlex 控制台](#)
 - [2.2 通过终端启动整机硬件](#)
 - [2.3 启动 VR 遥操作](#)
 - [2.4 快速检查](#)
- [第三章 全身管理工具](#)
 - [3.1 openflex_gui 整机管理 GUI](#)
 - [3.2 openflex_manager 整机电机配置工具](#)
 - [3.3 系统级常用维护命令](#)
- [第四章 全身运动控制系统](#)
 - [4.1 全身控制快速开始](#)
 - [4.2 整机层级与代码组织](#)
 - [4.3 整机 URDF 与 TF 树](#)
 - [4.4 整机 controller_manager 与控制器列表](#)
 - [4.5 CAN 总线总体规划](#)
- [第五章 全身VR遥操作系统](#)
 - [5.1 全身 VR 遥操作快速开始](#)
 - [5.2 总体架构](#)
 - [5.3 openflex_vr_bridge UDP 协议](#)
 - [5.4 双臂 VR IK 节点](#)
 - [5.5 头部 VR 节点](#)
 - [5.6 摇杆底盘控制节点](#)
 - [5.7 升降台按键控制节点](#)
 - [5.8 腰控位置闭环控制节点](#)
 - [5.9 VR 紧急停止与安全策略](#)
- [第六章 底盘子系统](#)
 - [6.1 底盘快速开始](#)
 - [6.2 机械结构与尺寸](#)
 - [6.3 URDF 模型结构](#)
 - [6.4 硬件接口与 CAN 通信](#)
 - [6.5 运动学算法](#)
 - [6.6 里程计计算](#)
 - [6.7 控制器参数详解](#)
- [第七章 升降台子系统](#)
 - [7.1 机械与硬件](#)
 - [7.2 CANopen 协议与对象字典](#)
 - [7.3 ros2_control 接口与控制器](#)

- [7.4 自动归零流程](#)
 - [7.5 状态、限位与服务接口](#)
- [第八章 双臂子系统](#)
 - [8.1 机械结构与电机配置](#)
 - [8.2 关节限位与运动学](#)
 - [8.3 ros2_control 硬件接口实现](#)
 - [8.4 MIT 与 CSP 控制模式](#)
 - [8.5 双臂前缀与 Bimanual 配置](#)
 - [8.6 夹爪映射](#)
 - [8.7 动态 KP/KD 参数](#)
- [第九章 头部子系统](#)
 - [9.1 机械与电机](#)
 - [9.2 URDF 链接结构](#)
 - [9.3 ros2_control 头部硬件接口](#)
 - [9.4 启动慢回零](#)
 - [9.5 头部相机与视频流](#)

第一章 构建工作空间

1.1 构建与安装

如果您使用的是官方预配置主机，可以跳过本章，直接进入 [第二章 快速开始](#)。

首先构建工作空间:

```
cd ~
mkdir openflex_all
cd openflex_all
mkdir openflex_maps openflex_models openflex_ws
git clone -b v1.0_basic https://github.com/OpenFlex-Wheeled-Humanoid/openflex_drivers.git
git clone -b v1.0_basic https://github.com/OpenFlex-Wheeled-Humanoid/openflex_vr_apk.git
cd openflex_ws
mkdir src
cd src
mkdir openflex_armx openflex_head openflex_integrated openflex_lift_slide
openflex_chassis
git clone -b v1.0_basic https://github.com/OpenFlex-Wheeled-Humanoid/OpenFlex.git
git clone -b v1.0_basic https://github.com/OpenFlex-Wheeled-Humanoid/openflex_EX0.git
cd openflex_armx
git clone -b v1.0_basic https://github.com/OpenFlex-Wheeled-Humanoid/openarmx_description.git
git clone -b v1.0_basic https://github.com/OpenFlex-Wheeled-Humanoid/openarmx_ros2.git
git clone -b v1.0_basic https://github.com/OpenFlex-Wheeled-Humanoid/openarmx_teleop_vr.git
git clone -b v1.0_basic https://github.com/OpenFlex-Wheeled-Humanoid/openarmx_tools.git
cd ..
cd openflex_chassis
git clone -b v1.0_basic https://github.com/OpenFlex-Wheeled-Humanoid/base_model_interface_layer.git
git clone -b v1.0_basic https://github.com/OpenFlex-Wheeled-Humanoid/hardware_sensor_layer.git
```

```
git clone -b v1.0_basic https://github.com/OpenFlex-Wheeled-Humanoid/mapping_localization_layer.git
git clone -b v1.0_basic https://github.com/OpenFlex-Wheeled-Humanoid/motion_control_layer.git
git clone -b v1.0_basic https://github.com/OpenFlex-Wheeled-Humanoid/navigation_layer.git
git clone -b v1.0_basic https://github.com/OpenFlex-Wheeled-Humanoid/system_bringup_layer.git
git clone -b v1.0_basic https://github.com/OpenFlex-Wheeled-Humanoid/tools_common_layer.git
cd ..
cd openflex_head
git clone -b v1.0_basic https://github.com/OpenFlex-Wheeled-Humanoid/openarmx_head_bringup.git
git clone -b v1.0_basic https://github.com/OpenFlex-Wheeled-Humanoid/openarmx_head_description.git
git clone -b v1.0_basic https://github.com/OpenFlex-Wheeled-Humanoid/openarmx_head_hardware.git
git clone -b v1.0_basic https://github.com/OpenFlex-Wheeled-Humanoid/openarmx_head_teleop_vr_pico.git
git clone -b v1.0_basic https://github.com/OpenFlex-Wheeled-Humanoid/openarmx_head_tools.git
git clone -b v1.0_basic https://github.com/OpenFlex-Wheeled-Humanoid/openarmx_head_visio_h264.git
cd ..
cd openflex_integrated
git clone -b v1.0_basic https://github.com/OpenFlex-Wheeled-Humanoid/openarmx_integrated_bringup.git
git clone -b v1.0_basic https://github.com/OpenFlex-Wheeled-Humanoid/openarmx_integrated_description.git
git clone -b v1.0_basic https://github.com/OpenFlex-Wheeled-Humanoid/openflex_gui.git
git clone -b v1.0_basic https://github.com/OpenFlex-Wheeled-Humanoid/openflex_manager.git
cd ..
cd openflex_lift_slide
git clone -b v1.0_basic https://github.com/OpenFlex-Wheeled-Humanoid/lift_slide_bringup.git
git clone -b v1.0_basic https://github.com/OpenFlex-Wheeled-Humanoid/lift_slide_description.git
git clone -b v1.0_basic https://github.com/OpenFlex-Wheeled-Humanoid/lift_slide_driver.git
git clone -b v1.0_basic https://github.com/OpenFlex-Wheeled-Humanoid/lift_slide_msgs.git
git clone -b v1.0_basic https://github.com/OpenFlex-Wheeled-Humanoid/lift_slide_panel.git
cd ..
git clone -b v1.0_basic https://github.com/OpenFlex-Wheeled-Humanoid/openflex_vr_bridge.git
python3 -m pip install openarmx_arm_driver -i
https://mirrors.tuna.tsinghua.edu.cn/pypi/web/simple
```

给脚本添加执行权限，先执行：

```
cd ~/openflex_all/openflex_ws/src/OpenFlex
chmod +x ./install_openflex_drivers_and_build.sh
```

执行安装脚本。该脚本会安装 `~/openflex_all/openflex_drivers` 中的本地依赖包，按依赖关系分组编译 ROS 包，并创建桌面控制台快捷方式。

```
cd ~/openflex_all/openflex_ws/src/OpenFlex
./install_openflex_drivers_and_build.sh
```

第二章 快速开始

2.1 通过桌面 GUI 启动 OpenFlex 控制台

首先右击桌面上的 **OpenFlex 控制台**，选择 允许运行。然后双击桌面上的 **OpenFlex 控制台**。如图所示：

(之后在这个地方插入图片)

也可以从终端启动：

```
cd ~/openflex_all/openflex_ws
source /opt/ros/humble/setup.bash
source install/setup.bash
ros2 run openflex_gui openflex_gui
```

2.2 通过终端启动整机硬件

启动前确认 6 路 CAN 已上电并且启动 CAN 口，默认映射为：右臂 **can0**、左臂 **can1**、头部 **can2**、升降台 **can3**、底盘驱动 **can4**、底盘转向 **can5**。

启动 CAN 口：(也可以在 **OpenFlex 控制台** 中点击 **启用全部 CAN**)

```
python3
~/openflex_all/openflex_ws/src/openflex_integrated/openflex_manager/scripts/en_all_can.py
```

关闭 CAN 口：(也可以在 **OpenFlex 控制台** 中点击 **禁用全部 CAN**)

```
python3
~/openflex_all/openflex_ws/src/openflex_integrated/openflex_manager/scripts/dis_all_can.py
```

检查电机状态：(也可以在 **OpenFlex 控制台** 中点击 **检查电机状态**)

```
python3
~/openflex_all/openflex_ws/src/openflex_integrated/openflex_manager/scripts/check_motor_status.py
```

启动整机控制：

```
cd ~/openflex_all/openflex_ws
source /opt/ros/humble/setup.bash
source install/setup.bash
ros2 launch openarmx_integrated_bringup integrated_robot_bringup.launch.py \
  use_fake_hardware:=false \
  chassis_steering_can:=can5 \
  chassis_driving_can:=can4 \
  left_arm_can:=can1 \
  right_arm_can:=can0 \
  lift_can:=can3 \
  lift_node_id:=16 \
  head_can:=can2 \
  use_rviz:=true
```

2.3 启动 VR 遥操作

✖ 在 Pico 上安装桥接软件 连接设备

1. 开启开发者模式并进入 USB 调试模式。
开启开发者模式：设置 > 关于本机 > 连续点击软件版本号
开启 USB 调试：设置 > 开发者选项 > USB 调试
2. 使用 USB Type-C 数据线将 Pico 连接到 PC。

安装 Pico 桥接 APK

```
# 安装 ADB 工具
sudo apt install adb

# 进入 APK 所在目录
cd ~/openflex_all/openflex_vr_apk/apk/pico

# 安装桥接软件
adb install OpenFlex.apk
```

启动 VR 遥操作：（默认不启用 VR 底盘速度控制）

```
cd ~/openflex_all/openflex_ws
source /opt/ros/humble/setup.bash
source install/setup.bash
ros2 launch openarmx_integrated_bringup integrated_vr_teleop.launch.py
vr_chassis:=false
```

如果需要启用 VR 底盘速度控制：

```
ros2 launch openarmx_integrated_bringup integrated_vr_teleop.launch.py
vr_chassis:=true
```

2.4 快速检查

```
ros2 control list_controllers
ros2 topic hz /joint_states
ros2 topic hz /pico_left_controller/pose
ros2 topic hz /pico_right_controller/pose
ip link show type can
```

第三章 全身管理工具

3.1 openflex_gui 整机管理 GUI

openflex_gui 是基于 PyQt5 的 OpenFlex 整机管理界面，用于集中完成 CAN 总线管理、电机状态检查、整机 ros2_control 启停、VR 遥操作启停和电量显示。它不是替代底层 launch 的独立控制器，而是把常用启动命令、CAN 辅助脚本和状态检查集中到一个桌面面板里。

代码文件位置： `src/openflex_integrated/openflex_gui/openflex_gui/main_window.py`

3.1.1 安装方式

安装脚本会创建桌面快捷方式：

```
cd ~/openflex_all/openflex_ws/src/OpenFlex
./install_openflex_drivers_and_build.sh
```

手动创建桌面快捷方式 GUI：

```
cat > ~/桌面/OpenFlex控制台.desktop << EOF
[Desktop Entry]
Version=1.0
Type=Application
Name=OpenFlex 控制台
Exec=bash -c "source /opt/ros/humble/setup.bash && source
\${HOME}/openflex_all/openflex_ws/install/setup.bash && ros2 run openflex_gui
openflex_gui"
Icon=\${HOME}/openflex_all/openflex_ws/src/openflex_integrated/openflex_gui/openflex_gui
/openflex_vr.svg
Terminal=false
Categories=Robotics;
EOF
chmod +x ~/桌面/OpenFlex控制台.desktop
```

3.1.2 启动方式

终端启动：

```
cd ~/openflex_all/openflex_ws
source /opt/ros/humble/setup.bash
source install/setup.bash
ros2 run openflex_gui openflex_gui
```

通过桌面快捷方式启动 GUI：快捷方式名称为 **OpenFlex 控制台**

3.1.3 功能区

GUI 当前包含 6 个功能区：

功能区	按钮 / 选项	实际作用
CAN 总线	启用全部 CAN、禁用全部 CAN	调用包内 helper 脚本配置 <code>can0~can5</code> ，波特率为 1 Mbps
电机状态	检查全部电机状态	通过 <code>openflex_manager/scripts</code> 中的 CAN 工具检查升降台、底盘、双臂、头部电机响应
整机控制	启动整机控制、停止	启动或停止 <code>openarmx_integrated_bringup integrated_robot_bringup.launch.py</code>
VR 遥操作	启动 VR 遥操作、停止、vr控制底盘速度	启动 <code>openarmx_integrated_bringup integrated_vr_teleop.launch.py</code> ；勾选底盘速度选项时传入 <code>vr_chassis:=true</code> ，否则传入 <code>vr_chassis:=false</code>
键盘底盘控制	启动键盘底盘控制、停止	启动 <code>swerve_bringup swerve_teleop.py</code> ，用于键盘控制底盘移动
电量显示	显示电量、停止	配置电池串口权限并启动 <code>openarmx_battery_monitor auto_pack_overlay.launch.py</code>

3.1.4 整机控制命令

点击 **启动整机控制** 时，GUI 执行的核心命令为：

```
ros2 launch openarmx_integrated_bringup integrated_robot_bringup.launch.py \
  use_fake_hardware:=false \
  chassis_steering_can:=can5 \
  chassis_driving_can:=can4 \
  left_arm_can:=can1 \
  right_arm_can:=can0 \
  lift_can:=can3 \
  lift_node_id:=16 \
  head_can:=can2 \
  use_rviz:=true
```

点击 **停止** 时，GUI 会先尝试将底盘控制器和底盘硬件切到 inactive：

```
ros2 control set_controller_state swerve_drive_controller inactive
ros2 control set_hardware_component_state swerve_drive_system inactive
```

然后再向整机 bringup 进程发送 SIGINT，超时后强制结束进程。

3.1.5 VR 启动逻辑

VR 功能区有两种启动路径：

vr控制 底盘速度	启动命令	说明
未勾选	<code>ros2 launch openarmx_integrated_bringup integrated_vr_teleop.launch.py vr_chassis:=false</code>	启动全身 VR 遥操作封装，适合常规 VR 操控
已勾选	<code>ros2 launch openarmx_integrated_bringup integrated_vr_teleop.launch.py vr_chassis:=true</code>	使用整机 bringup 中的 VR teleop launch，并启用 VR 底盘速度控制

GUI 会在启动整机控制后检查 `/controller_manager` 中关键控制器是否 `active`；就绪后可以继续启动 VR。当前检查项包括：

- `joint_state_broadcaster`
- `swerve_drive_controller`
- `velocity_controller`
- `left_forward_position_controller`
- `right_forward_position_controller`
- `head_forward_position_controller`

3.1.6 状态灯与日志

每个功能区左侧都有状态灯：

状态	含义
灰色	空闲或未启动
绿色	正常运行或操作成功
红色	操作失败、进程异常退出或状态检查异常

底部日志窗口会输出 helper 脚本、launch 进程和状态检查结果。出现异常时，应优先查看日志窗口中红色错误行。

3.2 openflex_manager 整机电机配置工具

`openflex_manager` 是 OpenFlex 整机管理工具包，提供 CAN 接口使能/关闭、电机状态检查等整机维护功能。

代码文件位置：`src/openflex_integrated/openflex_manager/`

3.2.1 主要功能

功能	说明
电机状态检查	查询升降台、底盘、双臂、头部电机是否在线
CAN 接口使能	一键 <code>ip link set canX up</code>
CAN 接口关闭	一键 <code>ip link set canX down</code>
配置管理	读取 OpenFlex 默认 CAN 映射与本地配置

3.2.2 整机默认 CAN 映射

以下映射是当前 OpenFlex 整机默认接线和启动参数，和 `openflex_gui`、`integrated_robot_bringup.launch.py`、整机电机管理脚本保持一致。注意双臂不是 `can0=左臂`、`can1=右臂`，当前默认是右臂在 `can0`，左臂在 `can1`。

CAN 口	波特率	用途
can0	1 Mbps	右臂
can1	1 Mbps	左臂
can2	1 Mbps	头部
can3	1 Mbps	升降台
can4	1 Mbps	底盘驱动 UM 轮毂电机
can5	1 Mbps	底盘转向 RS06

3.3 系统级常用维护命令

3.3.1 CAN 接口管理

```
# 上电单个 CAN (1 Mbps)
sudo ip link set can0 type can bitrate 1000000
sudo ip link set can0 up

# 上电全部 CAN (脚本)
for i in 0 1 2 3 4 5; do
    sudo ip link set can${i} type can bitrate 1000000 && sudo ip link set can${i} up
done

# 查看 CAN 状态
ip link show type can

# 查看 CAN 流量
candump can0
```

3.3.2 ROS2 系统检查

```
# 检查所有控制器状态
ros2 control list_controllers

# 关键话题频率
ros2 topic hz /joint_states
ros2 topic hz /odom
ros2 topic hz /cmd_vel

# TF 检查
ros2 run tf2_ros tf2_echo base_link lift_carriage_link

# 升降台服务
ros2 service call /lift_slide_driver/enable std_srvs/srv/Trigger
ros2 service call /lift_slide_driver/start_homing std_srvs/srv/Trigger

# 动态修改双臂 KP
ros2 param set /openarmx_left_hardware_params kp_joint1 30.0
ros2 param set /openarmx_right_hardware_params kp_joint1 30.0
```

3.3.3 VR 系统检查

```
# 检查 UDP 端口监听
sudo netstat -tunlp | grep 5100

# 检查 VR 话题
ros2 topic hz /pico_left_controller/pose
ros2 topic hz /pico_right_controller/pose
ros2 topic hz /pico_head/pose
```

3.3.4 整机故障排查顺序

1. source 环境
source /opt/ros/humble/setup.bash
source install/setup.bash
2. 检查 CAN 接口
ip link show type can
3. 检查电机响应
candump can0 | head
4. 检查控制器状态
ros2 control list_controllers
5. 检查关键话题
ros2 topic hz /joint_states
6. VR 输入检查
ros2 topic hz /pico_left_controller/pose

第四章 全身运动控制系统

4.1 全身控制快速开始

4.1.1 启动整机硬件

启动前确认 启动了6 路 CAN 并且已上电。默认映射：右臂 can0、左臂 can1、头部 can2、升降台 can3、底盘驱动 can4、底盘转向 can5。

```
cd ~/openflex_all/openflex_ws
source install/setup.bash
ros2 launch openarmx_integrated_bringup integrated_robot_bringup.launch.py \
  use_fake_hardware:=false \
  chassis_steering_can:=can5 \
  chassis_driving_can:=can4 \
  left_arm_can:=can1 \
  right_arm_can:=can0 \
  lift_can:=can3 \
  head_can:=can2
```

无真实硬件时可使用 mock 模式：

```
cd ~/openflex_all/openflex_ws
source install/setup.bash
ros2 launch openarmx_integrated_bringup integrated_robot_bringup.launch.py \
  use_fake_hardware:=true
```

4.1.2 启动后检查

```
ros2 control list_controllers
ros2 topic hz /joint_states
ros2 topic list | grep -E "cmd_vel|joint_states|controller|lift|head"
ros2 service list | grep lift_slide_driver
```

现象	判断
joint_state_broadcaster 为 active	整机关节状态广播正常
swerve_drive_controller 为 active	底盘控制器已加载
左/右臂 position controller 为 active	双臂控制器已加载
head_forward_position_controller 为 active	头部控制器已加载

4.2 整机层级与代码组织

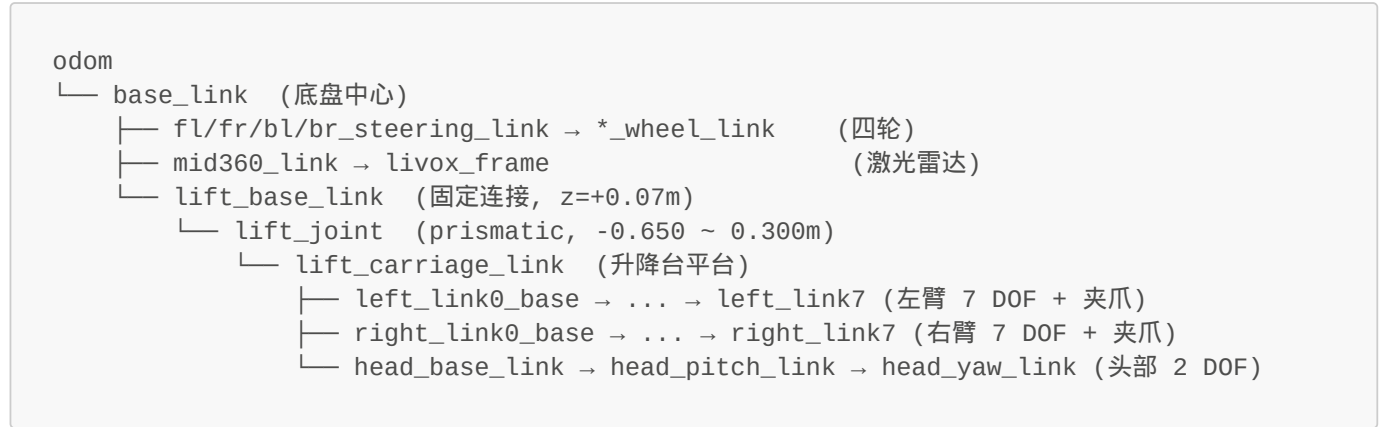
```
src/
├─ openflex_chassis/          ← 底盘子系统（第六章）
│   ├── base_model_interface_layer/swerve_description/
│   ├── hardware_sensor_layer/swerve_hardware/
│   ├── motion_control_layer/swerve_controller/
│   ├── system_bringup_layer/swerve_bringup/
│   ├── navigation_layer/swerve_navigation/
│   ├── navigation_layer/nmpc_controller/
│   └─ mapping_localization_layer/
├─ openflex_lift_slide/      ← 升降台子系统（第七章）
│   ├── lift_slide_description/
│   ├── lift_slide_driver/
│   ├── lift_slide_msgs/
│   └─ lift_slide_panel/
├─ openflex_armx/           ← 双臂子系统（第八章）
│   ├── openarmx_description/
│   ├── openarmx_ros2/openarmx_hardware/
│   ├── openarmx_teleop_vr/
│   └─ openarmx_tools/
├─ openflex_head/           ← 头部子系统（第九章）
│   ├── openarmx_head_description/
│   ├── openarmx_head_hardware/
│   ├── openarmx_head_teleop_vr_pico/
│   └─ openarmx_head_visio_h264/
├─ openflex_integrated/     ← 集成层
│   ├── openarmx_integrated_bringup/
│   ├── openarmx_integrated_description/
│   └─ openflex_gui/
└─ openflex_vr_bridge/      ← VR 手柄 UDP 桥接
```

4.3 整机 URDF 与 TF 树

整机 URDF:

src/openflex_integrated/openarmx_integrated_description/urdf/openarmx_integrated_robot.urdf.xacro

4.3.1 整机机械链路



设计要点： 双臂和头部均挂载在 lift_carriage_link 下，升降台运动会同时带动双臂和头部整体上下移动。

4.4 整机 controller_manager 与控制器列表

配置文件:

src/openflex_integrated/openarmx_integrated_bringup/config/integrated_controllers.yaml

整机所有子系统共用一个 controller_manager，更新频率 100 Hz。

控制器名	类型	管理的关节	说明
joint_state_broadcaster	JointStateBroadcaster	所有关节	广播到 /joint_states
swerve_drive_controller	SwerveDriveController	8 个底盘关节	底盘运动控制 + 里程计
lift_position_controller	ParamForwardingPositionController	lift_joint	升降台位置控制
lift_manual_position_controller	LiftSlideManualPositionController	lift_joint	升降台手动点动 + 步进位置控制
velocity_controller	ParamForwardingVelocityController	lift_joint	旧速度控制器，默认 inactive
left_forward_position_controller	JointGroupPositionController	8 个左臂关节	左臂位置控制
right_forward_position_controller	JointGroupPositionController	8 个右臂关节	右臂位置控制
head_forward_position_controller	ForwardCommandController	2 个头部关节	头部位置控制

命令话题

话题	消息类型	控制器
/cmd_vel	Twist	swerve_drive_controller

话题	消息类型	控制器
/lift_position_controller/commands	Float64MultiArray	lift_position_controller
/lift_manual_position_controller/jog_command	Float64	lift_manual_position_controller
/lift_manual_position_controller/step_command	Float64MultiArray	lift_manual_position_controller
/left_forward_position_controller/commands	Float64MultiArray	左臂
/right_forward_position_controller/commands	Float64MultiArray	右臂
/head_forward_position_controller/commands	Float64MultiArray	头部

4.5 CAN 总线总体规划

CAN 口	子系统	协议	波特率	电机/设备
can0	右臂	Robstride 自定义	1 Mbps	RS04×2 + RS03×2 + RS00×3 + RS00(夹爪)
can1	左臂	Robstride 自定义	1 Mbps	同上
can2	头部	Robstride 自定义	1 Mbps	RS00×2 (yaw + pitch)
can3	升降台	CANopen (CiA 402)	1 Mbps	节点 ID 16
can4	底盘驱动	CANopen (标准帧)	1 Mbps	UM 轮毂电机×4 (ID 1/2/3/4)
can5	底盘转向	RS06 自定义 (扩展帧)	1 Mbps	RS06×4 (ID 5/6/7/8)

开机 CAN 初始化：

```
for i in 0 1 2 3 4 5; do
    sudo ip link set can${i} type can bitrate 1000000 && sudo ip link set can${i} up
done
```

第五章 全身VR遥操作系统

5.1 全身 VR 遥操作快速开始

5.1.1 前置条件

先启动整机硬件控制链路：

```
cd ~/openflex_all/openflex_ws
source install/setup.bash
ros2 launch openarmx_integrated_bringup integrated_robot_bringup.launch.py \
    use_fake_hardware:=false \
    chassis_steering_can:=can5 \
    chassis_driving_can:=can4 \
    left_arm_can:=can1 \
    right_arm_can:=can0 \
    lift_can:=can3 \
    head_can:=can2
```

5.1.2 启动 VR 遥操作

```
cd ~/openflex_all/openflex_ws
source install/setup.bash
ros2 launch openarmx_integrated_bringup integrated_vr_teleop.launch.py
```

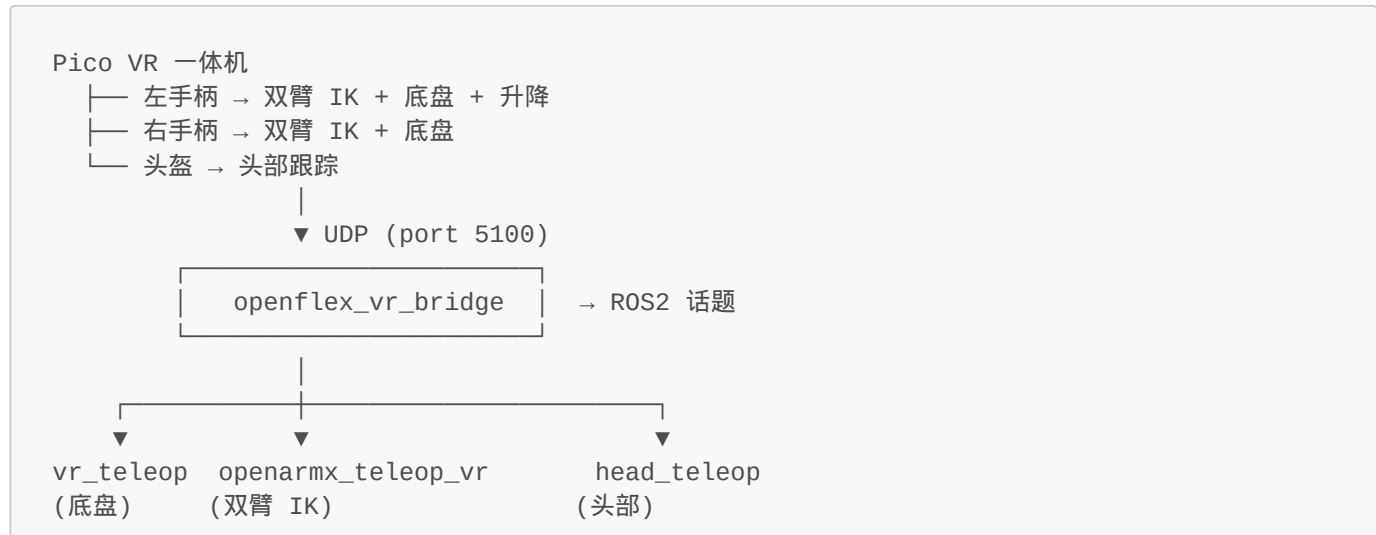
功能	参数	默认	说明
双臂 IK	—	始终启用	无法关闭
摇杆底盘	enable_joystick_control	true	左摇杆平移、右摇杆转向
按键升降	enable_button_lift	true	左手 X/Y 控制
头部跟踪	enable_head_teleop	true	VR 头盔控制头部
腰控	enable_waist_control	false	默认关闭

5.1.3 操作检查

```
ros2 topic hz /pico_left_controller/pose
ros2 topic hz /cmd_vel
ros2 topic hz /left_forward_position_controller/commands
ros2 topic hz /head_forward_position_controller/commands
```

操作	作用
左/右手柄移动	控制双臂末端位姿
手柄扳机/握持	控制夹爪开合
左摇杆	底盘前后和左右平移
右摇杆 X	底盘旋转
左手 X/Y	升降台下降/上升
头盔转动	头部 yaw/pitch 跟随

5.2 总体架构





VR Launch 文件位置：
`src/openflex_integrated/openarmx_integrated_bringup/launch/integrated_vr_teleop.launch.py`

5.3 openflex_vr_bridge UDP 协议

代码文件位置： `src/openflex_vr_bridge/src/pose_bridge_node.cpp`

参数	默认值	说明
监听地址	0.0.0.0	接收所有网络接口
监听端口	5100	UDP 端口

UDP 数据报格式

手柄完整数据包：

```
HAND LEFT x y z qx qy qz qw trigger grip button_a button_b button_x button_y
joystick_click joystick_x joystick_y rate timestamp_ns
```

增量更新包：

```
BTN LEFT/RIGHT A/B/X/Y/J pressed timestamp_ns
JOY LEFT/RIGHT joystick_x joystick_y timestamp_ns
TRIG LEFT/RIGHT trigger_value timestamp_ns
```

身体追踪器：

```
WAIST x y z qx qy qz qw timestamp_ns
HEAD x y z qx qy qz qw timestamp_ns
```

发布的 ROS2 话题

话题	类型	说明
<code>/pico_left_controller/pose</code>	PoseStamped	左手 6DOF 位姿
<code>/pico_left_controller/trigger</code>	Float32	扳机值 (0~1)
<code>/pico_left_controller/grip</code>	Float32	握力值 (0~1)
<code>/pico_left_controller/joystick_x</code>	Float32	摇杆 X 轴
<code>/pico_left_controller/joystick_y</code>	Float32	摇杆 Y 轴
<code>/pico_left_controller/rate</code>	Float32	速度模式

话题	类型	说明
/pico_head/pose	PoseStamped	头部位姿
/pico_tracker/waist/pose	PoseStamped	腰部位姿

5.4 双臂 VR IK 节点

代码文件位置：
src/openflex_armx/openarmx_teleop_vr/openarmx_teleop_vr/openarmx_teleop_vr_node.py

参数	默认值	说明
control_rate	100.0 Hz	IK 求解频率
grip_threshold	0.5	夹爪闭合阈值
resync_threshold_deg	5.0°	重新同步阈值
ik_iterations	3	IK 迭代次数
sync_joint_states_each_cycle	true	每周期同步关节状态
max_step_deg_joint1_2	8.0°/周期	1-2 关节最大步进
max_step_deg_joint3_4	5.0°/周期	3-4 关节最大步进
max_step_deg_joint5_7	5.0°/周期	5-7 关节最大步进

输出话题：

话题	数据
/left_forward_position_controller/commands	[j1..j7, finger]
/right_forward_position_controller/commands	[j1..j7, finger]

5.5 头部 VR 节点

代码文件位置：
src/openflex_head/openarmx_head_teleop_vr_pico/openarmx_head_teleop_vr_pico/head_teleop_node.py

参数	默认值	说明
slow_max_step_deg	2.0°/周期	慢速步进
fast_max_step_deg	5.0°/周期	快速步进
yaw_scale / pitch_scale	1.0	灵敏度
enable_soft_limits	true	启用软限位
startup_home_enabled	true	启动自动回零

控制逻辑：右手 Button B 切换相对控制开/关。开启时记录锚点四元数，计算相对旋转后提取 yaw/pitch，应用步进和软限位后发布。

5.6 摇杆底盘控制节点

代码文件位置：
src/openflex_chassis/system_bringup_layer/swerve_bringup/scripts/vr_teleop_node.py

输入	映射	范围
左手摇杆 Y	linear.x (前进/后退)	±max_linear_speed
左手摇杆 X	linear.y (左右横移)	±max_linear_speed
右手摇杆 X	angular.z (旋转)	±max_angular_speed
按键	功能	
右手 A	底盘启停切换	
右手 B	急停并禁用	

安全逻辑：超过 0.5 秒未收到 VR 数据 → 自动发送零速度。

5.7 升降台按键控制节点

代码文件位置：
src/openflex_chassis/system_bringup_layer/swerve_bringup/scripts/vr_lift_control_node.py

按键	功能
左手 X	下降点动
左手 Y	上升点动

输出：/lift_manual_position_controller/jog_command (Float64) 。按下时发布带符号的点动速度，松开、冲突、超时或急停时发布 0.0 停止；底层仍通过 LiftSlideManualPositionController 做连续位置目标规划。

5.8 腰控位置闭环控制节点

代码文件位置：
src/openflex_chassis/system_bringup_layer/swerve_bringup/scripts/waist_chassis_control_node.py

激活条件：左手或右手 trigger ≥ 0.5 时激活。

控制原理（增量目标位姿闭环）：

- 激活时记录锚点
- 计算腰部相对于锚点的增量 $\Delta x, \Delta y, \Delta yaw, \Delta lift$
- 闭环控制输出 → /cmd_vel + /lift_position_controller/commands

5.9 VR 紧急停止与安全策略

机制	说明
VR 信号丢失保护	底盘超 0.5s 无数据自动发零速
急停按键	右手 B 键底盘急停并禁用
步进限制	双臂和头部都有 max_step_deg
软限位	头部接近极限时缓冲减速

机制	说明
IK 关节限位裁剪	超出 URDF 限位的解被裁剪

第六章 底盘子系统

6.1 底盘快速开始

```
# 启动底盘运动控制
cd ~/openflex_all/openflex_ws
source install/setup.bash
ros2 launch swerve_bringup swerve_drive.launch.py

# 另开终端，键盘遥控
cd ~/openflex_all/openflex_ws
source install/setup.bash
ros2 run swerve_bringup swerve_teleop.py
```

参数	默认值	说明
use_rviz	true	是否启动 RViz
steering_can_interface	can5	转向 CAN
driving_can_interface	can4	驱动 CAN
max_wheel_speed	2.0	轮速上限 m/s
wheel_accel_limit	1.2	轮速变化率 m/s ²

键盘操作：**i**前进、**j**后退、**l**左转、**k**右转、**o**停止、**a**左平移、**d**右平移。

6.2 机械结构与尺寸

四轮独立转向独立驱动（Swerve Drive），每个轮组由 RS06 转向电机 + UM 轮毂伺服电机组成，共 8 个电机。

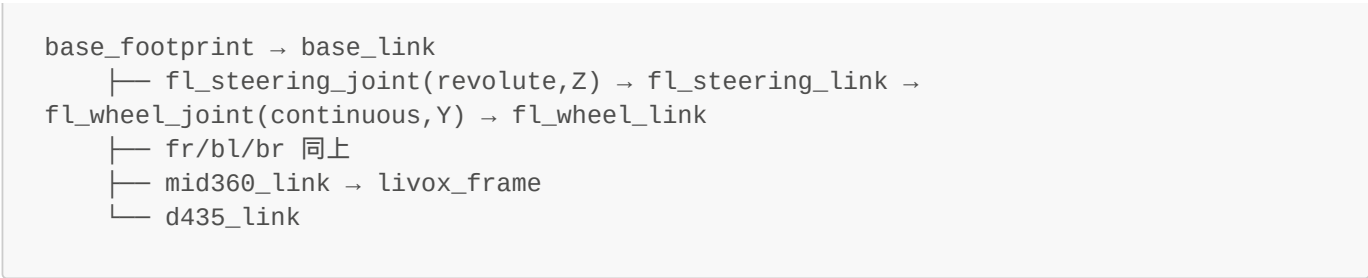
参数	值
底盘长×宽×高	0.50 × 0.40 × 0.14 m
轮距（Track Width）	0.52 m
轴距（Wheelbase）	0.42 m
车轮半径	0.14 m（外径） / 0.075 m（有效）
底盘质量	20.0 kg

轮组位置（相对 base_link）：FL(+0.21,+0.26)、FR(+0.21,-0.26)、BL(-0.21,+0.26)、BR(-0.21,-0.26)

6.3 URDF 模型结构

文件位置：

src/openflex_chassis/base_model_interface_layer/swerve_description/urdf/swerve.urdf.xacro

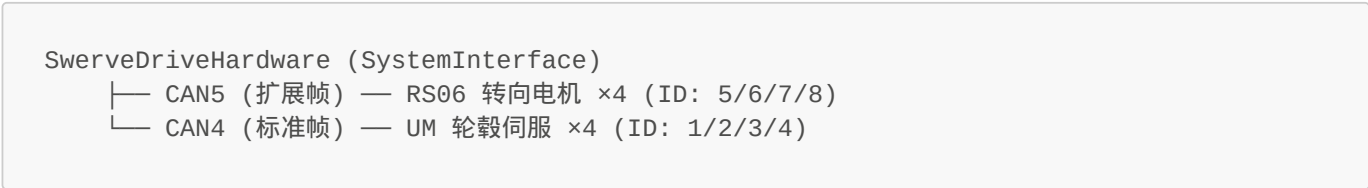


转向关节限位：±1.5708 rad (±90°)

6.4 硬件接口与 CAN 通信

代码文件位置：

src/openflex_chassis/hardware_sensor_layer/swerve_hardware/src/swerve_drive_hardware.cpp



RS06 初始化序列： MOTOR_STOP → RUN_MODE=5(CSP) → LIMIT_SPD=4.0 → LIMIT_CUR=8.0 → LOC_REF=zero → MOTOR_ENABLE

UM 驱动模式： Profile Velocity (PV)，CiA 402 状态机。控制字 0x0006 → 0x0007 → 0x000F 进入 Operation Enabled。

6.5 运动学算法

代码文件位置：

src/openflex_chassis/motion_control_layer/swerve_controller/src/swerve_drive_kinematics.cpp

逆运动学 (cmd_vel → 轮组状态)

```
module_vx_i = vx - ω × yi
module_vy_i = vy + ω × xi
目标角度_i = atan2(module_vy_i, module_vx_i)
目标轮速_i = sqrt(module_vx_i² + module_vy_i²)
```

关键优化

- 1. 角度锁定：cmd_vel 为零时使用上次命令角度，防止抖动
- 2. 角度差归一化：atan2(sin(diff), cos(diff)) 解决 ±n 边界抖动
- 3. 转向误差速度衰减：cos(error)^exponent 衰减，误差过大时置零
- 4. 加速度限幅：每周期 wheel_accel_limit × dt
- 5. 速度去饱和：超过 max_wheel_speed 时等比例缩小

6.6 里程计计算

正运动学通过最小二乘法从 4 个轮组状态反算 (vx, vy, ω)，二阶中点法积分位姿：

/

```
mid_heading = heading + omega*dt/2
x += (vx*cos(mid_heading) - vy*sin(mid_heading)) * dt
y += (vx*sin(mid_heading) + vy*cos(mid_heading)) * dt
```

发布话题： `/odom`，频率 50 Hz。

6.7 控制器参数详解

配置文件位置： `src/openflex_chassis/system_bringup_layer/swerve_bringup/config/`

参数	导航模式	建图模式	说明
<code>enable_odom_tf</code>	false	true	是否发布 odom → base_link TF
<code>max_wheel_speed</code>	1.2	2.0	轮速上限 m/s
<code>wheel_accel_limit</code>	1.0	1.2	加速度限制
<code>cmd_vel_timeout</code>	0.5	0.5	指令超时停车
<code>steering_align_threshold</code>	0.08	0.08	转向对齐阈值 rad
<code>steering_stop_threshold</code>	0.90	0.90	单轮停止驱动阈值

第七章 升降台子系统

7.1 机械与硬件

参数	值	说明
行程	-0.650 ~ 0.300 m	以 home 为零点的升降台软限位范围
安装偏移	z = +0.07 m	lift_base_link 相对 base_link
关节类型	prismatic（直线）	Z 轴方向
关节名	<code>lift_joint</code>	
通信接口	CAN3	CANopen 协议
节点 ID	16	
位置转换	2,000,000 counts/m	

文件位置：

- URDF： `src/openflex_lift_slide/lift_slide_description/urdf/lift_slide_module.urdf.xacro`
- 硬件接口：
`src/openflex_lift_slide/lift_slide_driver/src/lift_slide_hardware_interface.cpp`

7.2 CANopen 协议与对象字典

对象索引	名称	说明
0x6040	Controlword	控制字
0x6041	Statusword	状态字

对象索引	名称	说明
0x6060	Mode of Operation	运行模式
0x6064	Position Actual	当前位置
0x606C	Velocity Actual	当前速度
0x607A	Target Position	目标位置
0x60FF	Target Velocity	目标速度
0x60FD	Digital Inputs	限位开关

CiA 402 上电序列：Shutdown(0x0006) → Switch On(0x0007) → Enable Operation(0x000F)

7.3 ros2_control 接口与控制器

插件名： `lift_slide_driver/LiftSlideHardwareInterface`

状态接口： `position`, `velocity`, `cia402_state`, `statusword`, `is_enabled`, `is_fault`, `homing_state`, `homing_complete`, `upper/home/lower_limit_switch`

命令接口： `position`, `velocity`

控制器	类型	说明
<code>lift_manual_position_controller</code>	<code>LiftSlideManualPositionController</code>	默认激活，RViz 面板和 VR 按键点动用
<code>lift_position_controller</code>	<code>ParamForwardingPositionController</code>	位置控制，供腰控和直接位置命令使用
<code>velocity_controller</code>	<code>ParamForwardingVelocityController</code>	旧速度控制器，默认 inactive
<code>lift_state_controller</code>	<code>LiftSlideStateController</code>	状态发布、速度参数兼容话题

7.4 自动归零流程

升降台没有绝对编码器，每次上电需要归零建立位置参考。

参数	值
<code>homing_method</code>	27
<code>homing_speed</code>	0.010 m/s
<code>homing_timeout</code>	60.0 s
<code>home_switch_position_m</code>	0.650 m

在 `integrated_robot_bringup.launch.py` 中当 `auto_homing:=true` 时：

```
+9.0s   调用 /lift_slide_driver/enable → 使能
+12.0s  调用 /lift_slide_driver/start_homing → 归零
        └─ 检测到 home_switch → 停止 → 设定当前位置
```

7.5 状态、限位与服务接口

服务名	说明
/lift_slide_driver/enable	使能驱动器
/lift_slide_driver/start_homing	启动归零
/lift_slide_driver/return_home	返回归零位
/lift_slide_driver/quick_stop	紧急停止
话题	说明
/lift_slide_driver/motor_status	电机状态
/lift_slide_driver/homing_state	归零进度
/lift_slide_driver/limit_switch_state	限位开关

第八章 双臂子系统（OpenArmX）

8.1 机械结构与电机配置

每条手臂 7 个旋转关节 + 1 个夹爪，共 8 自由度。双臂合计 16 自由度。

关节	CAN ID	电机型号	说明
Joint 1	0x01	RS04	肩偏航
Joint 2	0x02	RS04	肩俯仰
Joint 3	0x03	RS03	上臂旋转
Joint 4	0x04	RS03	肘关节
Joint 5	0x05	RS00	前臂旋转
Joint 6	0x06	RS00	腕偏航
Joint 7	0x07	RS00	腕俯仰
夹爪	0x08	RS00	末端夹爪

CAN 分配：左臂 can1，右臂 can0

8.2 关节限位与运动学

关节	下限 (rad)	上限 (rad)	速度 (rad/s)
Joint 1	-1.25	3.0	10.47
Joint 2	-1.70	1.7	10.47
Joint 3	-1.57	1.57	10.47
Joint 4	0.0	1.8	10.47
Joint 5	-1.50	1.50	10.47
Joint 6	-0.75	0.75	10.47
Joint 7	-1.40	1.40	10.47

配置文件： `src/openflex_armx/openarmx_description/config/arm/v10/joint_limits.yaml`

8.3 ros2_control 硬件接口实现

插件名： `openarmx_hardware/OpenArmX_v10HW` 代码文件：
`src/openflex_armx/openarmx_ros2/openarmx_hardware/src/v10_simple_hardware.cpp`

回调	动作
<code>on_init()</code>	解析参数、创建 ROS2 参数节点（KP/KD）、初始化 CAN
<code>on_activate()</code>	使能所有电机；读取当前位置作为初始命令值
<code>on_deactivate()</code>	禁用所有电机
<code>read()</code>	读取位置/速度/力矩；应用方向乘数；夹爪 rad → m
<code>write()</code>	MIT: 发送 MotionControlParam；CSP: 发送 LOC_REF

8.4 MIT 与 CSP 控制模式

MIT 模式（默认）： 每周期发送 `{position, velocity=0, torque=0, kp, kd}`，可在线调节刚度/阻尼。

CSP 模式： 每周期仅发送目标位置 `LOC_REF`，驱动器内部闭环，轨迹更平滑。

8.5 双臂前缀与 Bimanual 配置

参数	左臂	右臂
<code>arm_prefix</code>	<code>left_</code>	<code>right_</code>
<code>can_interface</code>	<code>can1</code>	<code>can0</code>

关节名： `openarmx_{prefix}joint{N}`，如 `openarmx_left_joint1`

8.6 夹爪映射

夹爪 URDF 为 prismatic 关节，由旋转电机驱动：

方向	公式
关节 → 电机	$motor_rad = (joint_m / 0.044) \times 1.0472$
电机 → 关节	$joint_m = 0.044 \times (motor_rad / 1.0472)$

关节行程：0 ~ 0.044 m，电机行程：0 ~ 60°

8.7 动态 KP/KD 参数

MIT 模式下运行时可通过参数服务动态调节：

关节	默认 KP	默认 KD
Joint 1~4	50.0	2.5
Joint 5~7	10.0	0.5

关节	默认 KP	默认 KD
夹爪	50.0	2.5

```
ros2 param set /openarmx_left_hardware_params kp_joint1 30.0
ros2 param set /openarmx_right_hardware_params kp_joint1 30.0
ros2 param set /openarmx_left_hardware_params kd_joint5 1.0
ros2 param set /openarmx_right_hardware_params kd_joint5 1.0
```

场景	KP	KD	说明
VR 遥操作	50/10	2.5/0.5	高刚度跟踪
柔顺抓取	10/3	1.0/0.3	低刚度适应
力控实验	0	0.5~2.0	纯阻尼

第九章 头部子系统

9.1 机械与电机

2 自由度：俯仰（Pitch）+ 偏航（Yaw），挂载在 `lift_carriage_link` 上方。

关节	CAN ID	电机	安装位置
<code>openarmx_head_pitch_joint</code>	0x02	RS00	底部电机
<code>openarmx_head_yaw_joint</code>	0x01	RS00	顶部电机

通信接口：CAN2，默认 KP=100.0，KD=10.0

9.2 URDF 链接结构

文件：`src/openflex_head/openarmx_head_description/urdf/head.urdf.xacro`

```
head_base_link
├─ openarmx_head_pitch_joint (revolute, X轴, ±90°)
│   └─ head_pitch_link
│       └─ openarmx_head_yaw_joint (revolute, Z轴, ±90°)
│           └─ head_yaw_link (含相机)
```

9.3 ros2_control 头部硬件接口

插件名：`openarmx_head_hardware/OpenArmX_HeadHW` 代码文件：
`src/openflex_head/openarmx_head_hardware/src/head_hardware.cpp`

参数	默认值	说明
<code>can_interface</code>	can2	头部 CAN
<code>control_mode</code>	csp	mit / csp
<code>home_on_activate</code>	true	激活时自动慢回零

参数	默认值	说明
home_duration_sec	3.0	慢回零持续时间

控制器：`head_forward_position_controller`，命令话题
`/head_forward_position_controller/commands`，数据 `[yaw_rad, pitch_rad]`

9.4 启动慢回零（Soft Homing）

头部电机启动时执行慢回零，避免快速跳变：

```
on_activate() 时：
1. 读取当前位置 (yaw_curr, pitch_curr)
2. 在 home_duration_sec (3.0s) 内线性插值到零位
3. 完成后切换到正常控制模式
```

9.5 头部相机与视频流（H.264）

代码文件位置：`src/openflex_head/openarmx_head_visio_h264/`

当前 VR 视频协议：UDP + OAR3 分片格式，默认端口 5600，编码 H.264。

启动方式：

```
# 完整启动（相机 + VR 推流）
cd ~/openflex_all/openflex_ws
source install/setup.bash
ros2 launch openarmx_head_vision_h264 d435i_vr.launch.py

# 仅相机
cd ~/openflex_all/openflex_ws
source install/setup.bash
ros2 launch openarmx_head_vision_h264 d435i_source.launch.py

# 仅 VR 推流
cd ~/openflex_all/openflex_ws
source install/setup.bash
ros2 launch openarmx_head_vision_h264 vr_forwarder_only.launch.py \
  image_topic:=/vision/color/image_raw udp_port:=5600
```

预设模式	帧率	码率	适用
balanced	20 fps	4000 kbps	默认
high_quality	25 fps	6000 kbps	高画质
low_latency	25 fps	2500 kbps	极低延迟
bandwidth_saving	15 fps	1500 kbps	弱网

文档结束。传感器、建图、定位、导航、VLA 数据采集/训练/推理及算法原理详见高级篇。